kernel-sign

Release 5.0.0

Gene C

Feb 05, 2026

CONTENTS

1 kernel-sign **1**
- 1.1 Overview . . . 1
- 1.2 Available . . . 1

2 Introduction **3**
- 2.1 Kernel Modules . . . 3
- 2.2 How to sign kernel modules using a custom kernel . . . 3
- 2.3 Summary of what needs to be done . . . 4

3 Kernel configuration **5**
- 3.1 Kernel Config File . . . 5
- 3.2 Kernel command line . . . 5

4 Tools needed **7**
- 4.1 kernel build package . . . 7
- 4.2 genkey.py & x509.oot.genkey . . . 7
- 4.3 sign_module.py . . . 7
- 4.4 install-certs.py . . . 8
- 4.5 dkms support . . . 8

5 Modify PKGBUILD **9**
- 5.1 What to change . . . 9
- 5.2 prepare() . . . 9
- 5.3 _package-headers() . . . 9

6 Required Files **11**

7 Arch AUR packags **13**
- 7.1 AUR Packages . . . 13

8 License **15**

9 Older Changes **17**

10 License **19**

CHAPTER

ONE

KERNEL-SIGN

1.1 Overview

Signed kernel modules provide a mechanism for the kernel to verify the integrity of a module. This package provides the tools needed to build a kernel supporting signed *out of tree* (OOT) modules. An OOT module is any kernel module that is not part of the upstream Linux kernel itself.

All git tags are signed with arch@sapience.com key which is available via WKD or download from https://www.sapience.com/tech.

1.1.1 Latest Changes

Version 5.0

- Preliminary support for **ml-dsa-44**, **ml-dsa-65** and **ml-dsa-87**

  These are the NIST approved post quantum compiting algorithms.

  linux-next (as of 20260205) supports verifying signatures signed using ml-dsa-nn. This version of *kernel-sign* will generate keys for those ml-dsa-nn algorithms if specified in the kernel config.

  We rely on the signing tool that is provided by the kernel itself to sign any OOT kernel modules. Therefore, once the kernel tool supports signing with ml-dsa, then we will too.

- Note: A future version may require the *pyconcurrent* module.

  Its a lttle extra maintenance keeping duplicate code. For now, we will use the pyconcurrent module if available otherwise we use a local copy of the part we need.

- License updated to GPL version 2 or later.

- pkg-install script updated.

  byte compile the python code and remove any no-longer needed files.

1.2 Available

The current version of Kernel-Sign is in Github repo:

- Kernel_Sign - includes documentation pdf.

Also see:

- Kernel Docs - upstream kernel documentation on module signing.
- Arch Wiki - this may not keep be up to date.

CHAPTER

TWO

INTRODUCTION

2.1 Kernel Modules

The Linux kernel distinguishes and keeps separate the verification of modules from requiring or forcing modules to verify before allowing them to be loaded. Kernel modules fall into 2 classes:

Standard "in tree" modules which come with the kernel source code. They are compiled during the normal kernel build.

Out of tree modules which are not part of the kernel source distribution. They are built outside of the kernel tree, requiring the kernel headers package for each kernel they are to be built for. They can be built manually for a specific kernel and packaged, or they can be built whenever needed using DKMS DKMS

Examples of such packages, provided by Arch, include:

- Virtual gues modules virtualbox-guest-modules-arch

During a standard kernel compilation, the kernel build tools create a private/public key pair and sign every in tree module (using the private key). The public key is saved in the kernel itself. When a module is subsequently loaded, the public key can then be used to verify that the module is unchanged.

The kernel can be enabled to always verify modules and report any failures to standard logs. The choice to permit the loading and use of a module which could not be verified can be either compiled into kernel or turned on at run time using a kernel parameter as explained in Arch Wiki Kernel Parameters Arch Wiki Kernel Parameters.

2.2 How to sign kernel modules using a custom kernel

The starting point is based on building a custom kernel package outlined in Kernel/Arch Build System Kernel/Arch Build System,

We will adjust the build to:

- Sign the standard in tree kernel modules
- Provide what is needed to have signed out of tree modules and for the kernel to verify those modules.

Note: The goal is to have:

- In tree modules signed during the standard kernel build process.

  The standard kernel build creates a fresh public/private key pair on each build.

- Out of tree modules are signed and the associated public key is compiled in to the kernel. We will create a separate public/private key pair on each build.

2.3 Summary of what needs to be done

Each kernel build needs to made aware of the key/cert being used. Fresh keys are generated with each new kernel build.

A kernel config parameter is now used to make kernel aware of additional signing key:

```
CONFIG_SYSTEM_TRUSTED_KEYS="/path/to/oot-signing_keys.pem".
```

Keys and signing tools will be stored in current module build directory. Nothing needs to be done to clean this as removal is handled by the standard module cleanup.

Certs are thus installed in

```
/usr/lib/modules/<kernel-vers>-<build>/certs-local.
```

CHAPTER

THREE

KERNEL CONFIGURATION

3.1 Kernel Config File

CONFIG_SYSTEM_TRUSTED_KEYS will be added automatically as explained below. In addition the following config options should be set by either manually editing the 'config' file, or via make menuconfig in the linux 'src' area and subsequently copying the updated *.config* file back to the build file *config*. It is preferable to use elliptic curve type keys and zstd compression.

- CONFIG_MODULE_SIG=y

  Enable Loadable module suppot —>

  Module Signature Verification -> activate

- CONFIG_MODULE_SIG_FORCE=n

  Require modules to be validly signed -> leave off

  This allows the decision to enforce verified modules only as boot command line. If you are comfortable all is working then by all means change this to 'y' Command line version of this is : module.sig_enforce=1

- CONFIG_MODULE_SIG_HASH=sha512

  Automatically sign all modules -> activate Which hash algorithm -> SHA-512

  kernel 6.7 and later support sha3 hashes. The preferred hash choice is then sha3-512. This also requires openssl version 3.2 or newer.

- CONFIG_MODULE_COMPRESS_ZSTD=y

  Compress modules on installation -> activate Compression algorithm (ZSTD)

- CONFIG_MODULE_SIG_KEY_TYPE_ECDSA=y

  Cryptographic API —> Certificates for Signature Checking —> Type of module signing key to be generated -> ECDSA

- CONFIG_MODULE_ALLOW_MISSING_NAMESPACE_IMPORTS=n

  Enable Loadable module support —> Allow loading of modules with missing namespace imports -> set off

3.2 Kernel command line

After you are comfortable things are working well you can enable the kernel parameter to require that the kernel only permit verified modules to be loaded:

```
module.sig_enforce=1
```

CHAPTER

FOUR

TOOLS NEEDED

4.1 kernel build package

In the directory where the kernel package is built:

```
mkdir certs-local
```

This directory will provide the tools to create the keys, as well as signing kernel modules.

- Copy these files into certs-local directory:

```
genkeys.py
install-certs.py
sign_module.py
lib/*.py
x509.oot.genkey
```

4.2 genkey.py & x509.oot.genkey

genkey.py along with its configuration file x509.oot.genkey are used to create key pairs. It also provides the kernel with the key to sign out of tree modules by updating the config file used to build the kernel.

genkeys.py will create the key pairs in a directory named by date-time. It defaults to refreshing the keys every 7 days but this can be changed with the *–refresh* command line option.

It also creates a soft link named 'current' which points to the newly created directory with the 'current' keys. The actual key directory is named by date and time.

genkeys will check and update kernel configs given by the –config config(s) option. This takes either a single config file, or a shell glob for mulitple files. e.g. –config 'conf/config.*'. Remember to quote any wildcard characters to prevent the shell from expanding them.

All configs will be updated with the same key. The default keytype and hash are taken from the kernel config (from CONFIG_MODULE_SIG_HASH and CONFIG_MODULE_SIG_KEY_TYPE_xxx)[1].

If multiple kernel configs are being used, all must use same key and hash types.

4.3 sign_module.py

signs out of tree kernel modules. It can be run manually but is typically invoked by dkms/kernel-sign.sh. It handles modules compressed with zstd, xz and gzip and depends on python-zstandard package to help with those compressed with zstd.

[1] In earlier versions these defaulted to elliptic curve and sha512 and could be set from the command line.

4.4 install-certs.py

is called from the package_headers() function of PKGBUILD to install the signing keys. Example is given below.

These files are all provided.

4.5 dkms support

Important

DKMS a mechanism for out-of-tree modules to be compiled against the kernel headers. It is one thing to use signed modules provided in the kernel source but it is quite another to use modules, signed or not, that are out-of-tree. Any such module will *taint* the kernel. See kernel docs tainted_kernel for more information.

```
mkdir certs-local/dkms
```

and add 2 files to the dkms dir:

```
kernel-sign.conf
kernel-sign.sh
```

These will be installed in /etc/dkms and provide a mechanism for dkms to automatically sign modules using the local key discussed above - this is the reccommended way to sign kernel modules. As explained, below - once this is installed - all that is needed to have dkms automatically sign modules is to make a soft link:

```
cd /etc/dkms
ln -s kernel-sign.conf <module-name>.conf
```

For example: .. code-block:: bash

    ln -s kernel-sign.conf vboxdrv.conf

The link creation can easily be added to an arch package to simplify further if desired.

CHAPTER

FIVE

MODIFY PKGBUILD

5.1 What to change

We need to make changes to kernel build as follows:

5.2 prepare()

Add the following to the top of the prepare() function:

```
prepare() {
 ...
    echo "Rebuilding local signing key..."
    # adjust cerdir as needed
    certdir='../certs-local'
    $certdir/genkeys.py -v --config ../config --refresh 30d
  ...
}
```

The default key regeneration refresh period is 7 days, but this can be changed on the command line. If you want to create new keys monthly, then use "–refresh 30days" as an option to genekeys.py. You can refresh on every build by using "–refresh always". Refresh units can be seconds,minutes,hours,days or weeks.

5.3 _package-headers()

Add the following to the bottom of the _package-headers() function:

```
_package-headers() {

...

#
# Out of Tree Module signing
# This is run in the kernel source / build directory
#
echo "Local Signing certs for out of tree modules..."

certs_local_src="../../certs-local"
certs_local_dst="${builddir}/certs-local"

${certs_local_src}/install-certs.py $certs_local_dst
```

(continues on next page)

(continued from previous page)

```
# install dkms tools if needed
dkms_src="$certs_local_src/dkms"
dkms_dst="${pkgdir}/etc/dkms"
mkdir -p $dkms_dst

rsync -a $dkms_src/{kernel-sign.conf,kernel-sign.sh} $dkms_dst/
}
```

CHAPTER

SIX

REQUIRED FILES

This is the list of files referenced above. Remember to make scripts executable.

- certs-local/genkeys.py
- certs-local/install-certs.py
- certs-local/x509.oot.genkey
- certs-local/sign_module.py
- certs-local/lib/arg_parse.py
- certs-local/lib/refresh_needed.py
- certs-local/lib/class_genkeys.py
- certs-local/lib/get_key_hash.py
- certs-local/lib/make_keys.py
- certs-local/lib/signer_class.py
- certs-local/lib/update_config.py
- certs-local/lib/utils.py
- certs-local/dkms/kernel-sign.conf
- certs-local/dkms/kernel-sign.sh

CHAPTER

SEVEN

ARCH AUR PACKAGS

7.1 AUR Packages

There is an Arch Sign Modules package in the AUR along with its companion github repo Arch-SKM which make use of Kernel_Sign

arch-sign-modules reduces the manual steps for building a fully signed custom kernel to 3 commands to *Update*, *Build* and *Install* a kernel.

```
abk -u kernel-name
abk -b kernel-name
abk -i kernel-name
```

For more information see Arch-SKM-README and example Arch-SKM-PKGBUIILD

CHAPTER

EIGHT

LICENSE

CHAPTER

NINE

OLDER CHANGES

- Tidy and Improve code:
  - PEP-8, PEP-257, PEP-484 PEP-561
  - Refactor
- Add tests : run pytest in the *tests* directory.

  This will locate and use a kernel provided tool: /usr/lib/modules/xxx/build/scripts/sign-file
- key and hash types are now read from the kernel config file. Keeps everything consistent.
- Code re-org with supporing modules now moved to lib/xxx.py
- Code works with hash type sha3-xxx (e.g. *sha3-512*) available in kernel 6.7 and openssl 3.2 or later.

CHAPTER

TEN

LICENSE

The Linux kernel-sign tool facilitates signing out of tree kernel modules.

Copyright © 2022-present Gene C <arch@sapience.com>

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA.